TaxonVision-MLOps

Comprehensive Technical Implementation Guide

Systems Architecture, Infrastructure, and Codebase Reference

Benjamin Förster

October 11, 2026

Abstract

This technical report serves as an exhaustive, implementation-level architectural manual and code base reference for the TaxonVision-MLOps platform. Designed to document every structural component, configuration file, deployment manifest, mathematical loss formulation, and runtime daemon in the active repository, this document provides complete visibility into the physical server topology (`hive-mind` running Ubuntu 26.04.1 LTS, NVIDIA Driver 615.71.09, CUDA 13.4, and the NVIDIA GeForce RTX 5070 Ti with Blackwell SM 120), the lightweight Kubernetes (k3s) cluster partitioned into `jenkins-ci` and `taxon-vision` namespaces, the Cloudflare Zero Trust ingress tunnel, and the four-slot NVIDIA GPU time-slicing device plugin. Furthermore, this manual dissects the pure Pixi multi-environment toolchain, the automated DagsHub storage remote for Data Version Control (DVC) and MLflow tracking, the Cui et al. Class-Balanced Loss dynamics on biological long-tail Pareto distributions, the Optuna Bayesian hyperparameter optimisation engine, Split Conformal Prediction, Energy-Based Out-of-Distribution rejection, and the unified FastAPI and HTMX operator dashboard. Every configuration parameter, file extension, and container specification is strictly analysed based upon active codebase evidence.

Contents

Abstract **i**

1 Repository Architecture, Directory Hierarchy, and File Extension Semantics **1**
1.1 Root Directory Topology . . . 1
1.2 Configuration Manifests and Semantics . . . 1
1.3 Specialised File Formats and Storage Conventions . . . 1

2 Bare-Metal Server Environment and Pixi Toolchain Interlock **2**
2.1 Physical Compute Node Baseline . . . 2
2.2 Multi-Target Toolchain and Pixi Environment Isolation . . . 2
2.3 Command Orchestration via Justfile . . . 2

3 K3s Cluster Topology and Security RBAC **3**
3.1 Lightweight Kubernetes Architecture on Bare-Metal . . . 3
3.1.1 Embedded Datastore and Networking Subsystems . . . 3
3.1.2 Ingress Topology and Zero-Trust Tunnels . . . 4
3.2 Namespace Topology and Boundary Isolation . . . 4
3.3 Role-Based Access Control (RBAC) Specification . . . 4
3.3.1 ServiceAccount Scoping . . . 5
3.3.2 Role Rules and Least-Privilege Execution . . . 5
3.3.3 RoleBinding Guardrails . . . 5
3.4 Pod Security Standards and Admission Controls . . . 6
3.4.1 Non-Root Execution and File System Immutability . . . 6
3.4.2 Linux Capability Dropping . . . 6
3.4.3 Storage Volumes and Persistence Boundaries . . . 7

4 GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas **8**
4.1 The Single-GPU Multi-Tenancy Challenge . . . 8
4.2 NVIDIA Device Plugin Time-Slicing Configuration . . . 8
4.3 Deterministic VRAM Budget Capping and OOM Prevention . . . 8

5 Containerisation Architecture: Dockerfile and Composition Manifests **9**
5.1 Multi-Stage Container Construction Strategy . . . 9
5.2 Auxiliary Services and Docker Compose Topology . . . 9
5.3 Container Hardening and Privilege Minimisation . . . 9

6 Kubernetes Workloads, Service Manifests, and Rolling Updates **10**
6.1 API Deployment Specification . . . 10
6.2 Health Probes and Zero-Downtime Rolling Restarts . . . 10
6.3 Kubernetes Service Abstraction and Routing . . . 10

7 Declarative Jenkins Pipeline and Ephemeral Kubernetes Agents **11**
7.1 Declarative Pipeline Architecture and Pod Template . . . 11
7.2 Ephemeral Agent Lifecycle and Cache Volume Mounting . . . 11

7.3 Stage-by-Stage Execution and Promotion Gates 11

8 Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits 12
8.1 iNaturalist API Ingestion and Query Mechanics 12
8.2 Zero-Copy Apache Arrow Streaming Pipeline 12
8.3 Open License Verification and Attribution Retention 12

9 Data Version Control (DVC) and Remote Storage Topology 13
9.1 DVC Pipeline Graph and Cryptographic Manifests 13
9.2 DagsHub S3-Compatible Remote Storage Architecture 13
9.3 Synchronization Lifecycles in CI/CD and Local Workstations 13

10 Domain Modelling, DarwinCore Standards, and Taxonomic Trees 14
10.1 DarwinCore Taxon Schema and Catalog Implementation 14
10.2 Observation Entities and Spatiotemporal Metadata 14
10.3 License Verification Domain Invariants 14

11 Vision Backbone Factory and Representation Extraction Caching 15
11.1 Foundation Vision Backbone Portfolio 15
11.2 Extensible Feature Extractor Factory Architecture 15
11.3 Representation Extraction and Disk-Caching Strategy 15

12 Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics 16
12.1 The Biological Long-Tail Pareto Distribution 16
12.2 Effective Number of Samples Mathematical Derivation 16
12.3 PyTorch Implementation and Numerical Stabilisation 16

13 Classification Head Architecture, Optimization Loops, and Callbacks 17
13.1 Classification Head Geometry and Transfer Learning 17
13.2 Cached Embedding Training Loop and Mini-Batch Dynamics 17
13.3 Convergence Callbacks and Best-Weight Restoration 17

14 Bayesian Hyperparameter Optimization with Optuna TPE and ASHA 18
14.1 Bayesian Optimization Principles and TPE Mechanics 18
14.2 Search Space Parameterisation and Objective Formulation 18
14.3 Asynchronous Successive Halving (ASHA) Pruning in Practice 18

15 Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry 19
15.1 Experiment Tracking Architecture on DagsHub 19
15.2 Deterministic Lifecycle Management via RAII Scopes 19
15.3 Cryptographic Lineage Capture and Automated Model Promotion 19

16 Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection 20
16.1 The Epistemic Safety Problem in Biological Vision 20
16.2 Split Conformal Prediction and Statistical Coverage Bounds 20
16.3 Helmholtz Free Energy Out-of-Distribution Detection 20

17 Active Learning Strategy and Priority Triage Queue 21
17.1 Uncertainty-Based Query Acquisition Mechanics 21
17.2 Priority Triage Queue Implementation 21
17.3 Closing the Continuous Retraining Annotation Loop 21

18 ONNX Export, Zero-Allocation Runtime, and INT8 Quantization 22
18.1 Computational Graph Export and Dynamic Axes 22
18.2 Zero-Allocation C++ Inference Execution Engine 22
18.3 INT8 Quantization Calibration and Class Activation Mapping 22

19 FastAPI REST Service and Server-Rendered HTMX Operator Dashboard 23
19.1 Unified REST and Web Application Architecture 23
19.2 API Schemas, Endpoints, and Inference Routing 23
19.3 Server-Rendered HTMX Operator Interface 23

20 Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection 24
20.1 Prometheus Telemetry Instrumentation and Metrics Design 24
20.2 Serving Latency and Conformal Set Monitoring 24
20.3 Embedding Representation Drift via Wasserstein and MMD 24

21 Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets 25
21.1 Testing Architecture and Test Matrix Topology 25
21.2 Mathematical Invariant Assertions and Property-Based Testing 25
21.3 Cognitive Complexity Budgets and Pre-Commit Quality Gates 25

22 Open Knowledge Format (OKF v0.2) and Master Agent Routing 26
22.1 Open Knowledge Format (OKF v0.2) Schema and Trust Tiers 26
22.2 Multi-Agent Governance and Decision Routing Matrices 26
22.3 Automated Validation Scripts and D3 Knowledge Graph Visualization 26

23 Developer Operations, Environment Setup, and Engineering Runbooks 27
23.1 Developer Workstation Onboarding and Environment Setup 27
23.2 Dataset Synchronization and Remote Storage Runbook 27
23.3 Cluster Operations, GPU Debugging, and Emergency Rollback 27

Chapter 1

Repository Architecture, Directory Hierarchy, and File Extension Semantics

1.1 Root Directory Topology

1.2 Configuration Manifests and Semantics

1.3 Specialised File Formats and Storage Conventions

Chapter 2

Bare-Metal Server Environment and Pixi Toolchain Interlock

2.1 Physical Compute Node Baseline

2.2 Multi-Target Toolchain and Pixi Environment Isolation

2.3 Command Orchestration via Justfile

Chapter 3

K3s Cluster Topology and Security RBAC

The deterministic execution of the TaxonVision-MLOps pipeline necessitates a robust, secure, and low-latency orchestration layer. To avoid the virtualisation overhead inherent in public cloud providers, the platform employs a bare-metal Kubernetes architecture. This chapter strictly documents the topology of the lightweight Kubernetes (k3s) cluster, the explicit namespace boundaries, the Role-Based Access Control (RBAC) schemas, and the Pod Security Standards that enforce strict isolation between the continuous integration (CI) controllers, the machine learning experimentation tracking servers, and the operator-facing inference workloads.

3.1 Lightweight Kubernetes Architecture on Bare-Metal

In order to harmonise the computational intensity of Vision Foundation Model (VFM) training with the high-availability requirements of model serving, the deployment relies upon a single-node control plane and worker topology running natively on the bare-metal `hive-mind` host. The system employs *k3s*, a highly conformant and aggressively streamlined Kubernetes distribution developed by Rancher. K3s achieves a dramatically reduced memory footprint by stripping out legacy, out-of-tree cloud provider binaries and consolidating the core Kubernetes control plane components—namely the API server, the controller manager, and the scheduler—into a single, statically compiled binary process.

3.1.1 Embedded Datastore and Networking Subsystems

A conventional Kubernetes cluster relies upon a distributed *etcd* key-value store to maintain cluster state. Given the single-node physical architecture of `hive-mind`, deploying a quorum-based raft consensus algorithm introduces unnecessary network round-trip latencies and disk I/O overhead. Consequently, k3s is configured to utilise its embedded SQLite datastore. The integration of SQLite provides ACID-compliant transactional guarantees whilst operating entirely through local Unix sockets, thereby circumventing the TCP/IP stack and drastically accelerating control plane responsiveness during high-frequency ephemeral pod churn.

Network routing across the containerized workloads is mediated by the Flannel Container Network Interface (CNI). Flannel is configured to operate in Virtual eXtensible Local Area Network (VXLAN) mode. This establishes a layer-2 overlay network that encapsulates inter-pod traffic within UDP packets, facilitating seamless communication between the independent namespaces without exposing internal IP ranges to the bare-metal host's physical network interface. Furthermore, internal service discovery is resolved via the embedded CoreDNS controller, which dynamically provisions A and SRV records for the headless services associated with the MLflow tracking server and the TaxonVision API.

3.1.2 Ingress Topology and Zero-Trust Tunnels

The k3s installation is explicitly provisioned with the default Traefik ingress controller disabled. By bypassing standard Kubernetes Ingress objects, the architecture strictly eliminates the requirement for host port forwarding or external NodePort exposures that would otherwise compromise the bare-metal firewall boundary.

Instead, external routing is governed entirely by Cloudflare Zero-Trust Tunnels. A lightweight *cloudflared* daemon operates within the `kube-system` namespace, establishing persistent, outbound-only multiplexed connections to the Cloudflare global edge network. This design ensures that the `hive-mind` host exposes zero inbound listening ports to the public internet. Access to the TaxonVision-MLOps inference service is consequently masked behind an identity-aware Web Application Firewall (WAF), whilst internal CI/CD webhook deliveries are cryptographically validated at the Cloudflare edge prior to ingress.

3.2 Namespace Topology and Boundary Isolation

The principle of least privilege dictates that heterogeneous workloads must never share a flat execution environment. The k3s cluster is therefore partitioned into strictly segregated logical boundaries known as namespaces. This topology prevents the stochastic failures or resource exhaustion of one application from cascading into adjacent systems.

The cluster enforces the following explicit namespace segmentation:

- **`default`:** Retained as an empty baseline. Active deployment into the default namespace is prohibited by administrative policy. All TaxonVision serving workloads must operate outside this generic boundary.
- **`jenkins-ci`:** Dedicated to the continuous integration lifecycle, this boundary isolates the declarative Jenkins controller and its ephemeral agent pods. The namespace handles high-churn workflows including Optuna Bayesian hyperparameter optimisation, type checking, and PyTorch model training.
- **`mlflow`:** Dedicated to the MLflow tracking server and its backing PostgreSQL database. This namespace persists the metadata lineage of experimental trials, hyperparameter registries, and the Data Version Control (DVC) artefact references.
- **`taxon-vision`:** This namespace encapsulates the mission-critical, operator-facing inference workloads. It hosts the FastAPI deployment that serves the quantified ONNX computational graphs. Resource quotas are strictly allocated to guarantee low-latency HTTP responses, explicitly isolating the serving pipeline from computationally greedy background tasks.
- **`kube-system`:** Reserved exclusively for k3s core infrastructure controllers, including the NVIDIA time-slicing device plugin, CoreDNS, and the *cloudflared* tunnel daemon. Application workloads are cryptographically prohibited from scheduling within this domain.

Network Policies are provisioned to enforce cross-namespace constraints. For instance, the ephemeral `jenkins-ci` agents are permitted outbound access to the internet to retrieve dataset manifests from the DagsHub remote, yet they are strictly barred from initiating inbound TCP connections to the `taxon-vision` serving workload, mitigating the risk of lateral movement during a CI pipeline compromise.

3.3 Role-Based Access Control (RBAC) Specification

The security posture of the CI/CD pipeline is codified within the `deploy/k8s/jenkins-agent-rbac.yaml` manifest. Because the Jenkins controller requires the capacity to dynamically provision and terminate ephemeral machine learning runner pods, it necessitates elevated API permissions. However, granting cluster-admin privileges to the CI controller represents a catastrophic security vulnerability.

Therefore, the architecture employs a rigorous Role-Based Access Control (RBAC) schema that strictly bounds the operational scope of the `jenkins-agent`.

3.3.1 ServiceAccount Scoping

The baseline identity is established via a dedicated Kubernetes ServiceAccount. The manifest provisions the `jenkins-agent` identity exclusively within the `jenkins-ci` namespace, ensuring that any API token derived from this account possesses no inherent global authority.

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins-agent
  namespace: jenkins-ci
```

Listing 1: ServiceAccount definition for the Jenkins ephemeral agents.

3.3.2 Role Rules and Least-Privilege Execution

The operative permissions are defined within the `jenkins-agent-role`. This Role restricts the CI agent strictly to the manipulation of pod lifecycles and logging streams. Crucially, the `apiGroups` parameter is restricted to the core group (`""`), preventing the agent from modifying Deployments, DaemonSets, or StatefulSets.

The `resources` array is explicitly constrained to `pods`, `pods/exec`, and `pods/log`. The verbs granted—comprising `get`, `list`, `watch`, `create`, `delete`, `patch`, and `update`—are the absolute minimum required by the Jenkins Kubernetes plugin to provision the `ml-runner` containers, stream the standard output of the PyTorch training loops, and cleanly terminate the pod upon pipeline completion. The manifest furthermore explicitly authorises read-only access (`get`, `list`, `watch`) to `events`, enabling pipeline diagnostics to monitor scheduling latency and container image pull failures without exposing administrative cluster states. The role further dictates minimum privileges, enabling the ephemeral workers to manage their execution footprint via `pods`, `pods/exec`, and `pods/log` (facilitated by get, list, watch, create, delete, and deletecollection commands) whilst strictly isolating persistent data layers by merely granting read permissions on `persistentvolumeclaims` (get, list, watch).

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: jenkins-agent-role
  namespace: jenkins-ci
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/exec", "pods/log"]
    verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]
  - apiGroups: [""]
    resources: ["events"]
    verbs: ["get", "list", "watch"]
```

Listing 2: Role specification bounding API permissions to the minimum necessary pod operations.

3.3.3 RoleBinding Guardrails

To operationalise the permissions, a `RoleBinding` securely associates the `jenkins-agent` ServiceAccount with the `jenkins-agent-role`. Because this is a `RoleBinding` and not a `ClusterRoleBinding`, the permissions are strictly confined to the `jenkins-ci` namespace. This serves as a critical containment mechanism: even in the event of a severe container escape vulnerability within the `ml-runner`

environment, the compromised agent token cannot be leveraged to inspect secrets, modify workloads, or execute lateral movement into the `kube-system` or `taxon-vision` namespaces.

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: jenkins-agent-binding
  namespace: jenkins-ci
subjects:
  - kind: ServiceAccount
    name: jenkins-agent
    namespace: jenkins-ci
roleRef:
  kind: Role
  name: jenkins-agent-role
  apiGroup: rbac.authorization.k8s.io
```

Listing 3: RoleBinding binding the identity to the constrained permissions within the namespace.

The architecture further supplements the CI pipeline with a distinct `jenkins-deployer-role` within the `taxon-vision` namespace, permitting the continuous delivery phase to roll out updates to the API deployment. However, this cross-namespace capability is equally restricted, permitting modifications exclusively to the `deployments` and `services` objects required for the zero-downtime rolling update strategy.

3.4 Pod Security Standards and Admission Controls

Beyond the API boundary enforced by RBAC, the execution environment of the containers themselves is hardened using native Kubernetes Pod Security Contexts. By default, Linux containers execute with root privileges (`UID 0`), which exposes the underlying host kernel to severe risks in the event of a container breakout. The TaxonVision-MLOps workloads explicitly reject this paradigm.

3.4.1 Non-Root Execution and File System Immutability

The `taxon-vision-api` deployment enforces non-root execution via strict security contexts. The manifest explicitly sets `runAsNonRoot: true` and forces the process to execute under a heavily restricted user identity (`runAsUser: 10001`). This ensures that even if an attacker successfully exploits a vulnerability within the FastAPI Python runtime or the ONNX C++ bindings, the resulting shell is entirely unprivileged and unable to manipulate the container's root file system or interact with privileged host device nodes. Similarly, the ephemeral Jenkins agents execute under the constrained user identity `runAsUser: 1000`.

To further compound the difficulty of exploitation, workloads are intended to operate with a read-only root filesystem. By remounting the container's root directory as immutable, the architecture neutralises entire classes of post-exploitation persistence mechanisms, as malware or unauthorised binary drops cannot be written to typical execution paths such as `/bin` or `/usr/lib`.

3.4.2 Linux Capability Dropping

Linux capabilities partition the monolithic privileges of the root user into discrete, granular units. Standard Docker and Kubernetes container runtimes grant a default set of capabilities (such as `CAP_CHOWN` or `CAP_NET_BIND_SERVICE`) which are frequently superfluous for machine learning workloads. The deployment manifests aggressively drop all capabilities (`drop: ["ALL"]`), retaining only the absolute minimum required syscalls to operate the web server and the PyTorch computation graph. This drastically reduces the attack surface of the Linux kernel exposed to the containerized application.

3.4.3 Storage Volumes and Persistence Boundaries

Data persistence and caching are handled with equal rigor. The architecture minimises the usage of `hostPath` mounts, which inherently bridge the namespace isolation by directly mapping bare-metal file system directories into the container. When `hostPath` mounts are absolutely necessary—such as the exposure of the NVIDIA device nodes for time-slicing GPU virtualisation—they are strictly audited and mounted read-only where mathematically feasible.

Standard persistence is delegated to PersistentVolumeClaims (PVCs) establishing strict security boundaries. The architecture ensures that data access is rigorously constrained. By isolating the stateful volumes from the ephemeral compute logic, the k3s cluster topology achieves high physical determinism whilst preserving an aggressive, mathematically rigorous security posture. This isolation effectively restricts the potential blast radius of any individual workload, mitigating the risk of unauthorised data manipulation across administrative domains.

Chapter 4

GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas

4.1 The Single-GPU Multi-Tenancy Challenge

4.2 NVIDIA Device Plugin Time-Slicing Configuration

4.3 Deterministic VRAM Budget Capping and OOM Prevention

Chapter 5

Containerisation Architecture: Dockerfile and Composition Manifests

5.1 Multi-Stage Container Construction Strategy

5.2 Auxiliary Services and Docker Compose Topology

5.3 Container Hardening and Privilege Minimisation

Chapter 6

Kubernetes Workloads, Service Manifests, and Rolling Updates

6.1 API Deployment Specification

6.2 Health Probes and Zero-Downtime Rolling Restarts

6.3 Kubernetes Service Abstraction and Routing

Chapter 7

Declarative Jenkins Pipeline and Ephemeral Kubernetes Agents

7.1 Declarative Pipeline Architecture and Pod Template

7.2 Ephemeral Agent Lifecycle and Cache Volume Mounting

7.3 Stage-by-Stage Execution and Promotion Gates

Chapter 8

Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits

8.1 iNaturalist API Ingestion and Query Mechanics

8.2 Zero-Copy Apache Arrow Streaming Pipeline

8.3 Open License Verification and Attribution Retention

Chapter 9

Data Version Control (DVC) and Remote Storage Topology

9.1 DVC Pipeline Graph and Cryptographic Manifests

9.2 DagsHub S3-Compatible Remote Storage Architecture

9.3 Synchronization Lifecycles in CI/CD and Local Workstations

Chapter 10

Domain Modelling, DarwinCore Standards, and Taxonomic Trees

10.1 DarwinCore Taxon Schema and Catalog Implementation

10.2 Observation Entities and Spatiotemporal Metadata

10.3 License Verification Domain Invariants

Chapter 11

Vision Backbone Factory and Representation Extraction Caching

11.1 Foundation Vision Backbone Portfolio

11.2 Extensible Feature Extractor Factory Architecture

11.3 Representation Extraction and Disk-Caching Strategy

Chapter 12

Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics

12.1 The Biological Long-Tail Pareto Distribution

12.2 Effective Number of Samples Mathematical Derivation

12.3 PyTorch Implementation and Numerical Stabilisation

Chapter 13

Classification Head Architecture, Optimization Loops, and Callbacks

13.1 Classification Head Geometry and Transfer Learning

13.2 Cached Embedding Training Loop and Mini-Batch Dynamics

13.3 Convergence Callbacks and Best-Weight Restoration

Chapter 14

Bayesian Hyperparameter Optimization with Optuna TPE and ASHA

14.1 Bayesian Optimization Principles and TPE Mechanics

14.2 Search Space Parameterisation and Objective Formulation

14.3 Asynchronous Successive Halving (ASHA) Pruning in Practice

Chapter 15

Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry

15.1 Experiment Tracking Architecture on DagsHub

15.2 Deterministic Lifecycle Management via RAII Scopes

15.3 Cryptographic Lineage Capture and Automated Model Promotion

Chapter 16

Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection

16.1 The Epistemic Safety Problem in Biological Vision

16.2 Split Conformal Prediction and Statistical Coverage Bounds

16.3 Helmholtz Free Energy Out-of-Distribution Detection

Chapter 17

Active Learning Strategy and Priority Triage Queue

17.1 Uncertainty-Based Query Acquisition Mechanics

17.2 Priority Triage Queue Implementation

17.3 Closing the Continuous Retraining Annotation Loop

Chapter 18

ONNX Export, Zero-Allocation Runtime, and INT8 Quantization

18.1 Computational Graph Export and Dynamic Axes

18.2 Zero-Allocation C++ Inference Execution Engine

18.3 INT8 Quantization Calibration and Class Activation Mapping

Chapter 19

FastAPI REST Service and Server-Rendered HTMX Operator Dashboard

19.1 Unified REST and Web Application Architecture

19.2 API Schemas, Endpoints, and Inference Routing

19.3 Server-Rendered HTMX Operator Interface

Chapter 20

Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection

20.1 Prometheus Telemetry Instrumentation and Metrics Design

20.2 Serving Latency and Conformal Set Monitoring

20.3 Embedding Representation Drift via Wasserstein and MMD

Chapter 21

Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets

21.1 Testing Architecture and Test Matrix Topology

21.2 Mathematical Invariant Assertions and Property-Based Testing

21.3 Cognitive Complexity Budgets and Pre-Commit Quality Gates

Chapter 22

Open Knowledge Format (OKF v0.2) and Master Agent Routing

22.1 Open Knowledge Format (OKF v0.2) Schema and Trust Tiers

22.2 Multi-Agent Governance and Decision Routing Matrices

22.3 Automated Validation Scripts and D3 Knowledge Graph Visualization

Chapter 23

Developer Operations, Environment Setup, and Engineering Runbooks

23.1 Developer Workstation Onboarding and Environment Setup

23.2 Dataset Synchronization and Remote Storage Runbook

23.3 Cluster Operations, GPU Debugging, and Emergency Rollback

Bibliography

[1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[2] Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse gradient embeddings. In *International Conference on Learning Representations (ICLR)*, 2020.

[3] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9268–9277. IEEE, 2019.

[4] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xinwang Wang. A survey of deep active learning. *ACM Computing Surveys (CSUR)*, 54(9):1–40, 2021.

[5] Samuel Stevens, Jiaman Wu, Matthew J. Thompson, Elizabeth G. Campolongo, Chan Hee Song, David E. Carlyn, Li Dong, Wasila M. Dahdul, Charles Stewart, Tanya Berger-Wolf, Wei-Lun Chao, and Yu Su. Bioclip: A vision foundation model for the tree of life. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19412–19424. IEEE, 2024.

[6] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The inaturalist species classification and detection dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8769–8778. IEEE, 2018.